GUIA VISUAL 054

READ, ECHO E PRINTF

054

GUIAS LINUX

O relatório saía com três colunas alinhadas na máquina de quem escreveu. Chamado pelo `cron` com `sh inventario.sh`, virou `-e Servidor\tCPU\tMemoria`. O culpado é o `echo` — o único comando do shell cuja portabilidade é problema real.

O `ECHO` MENTE · O `PRINTF` NÃO

`echo -e "a\tb"`
Três implementações discordam: o **dash** (o `/bin/sh` do Kali) *sempre* interpreta a barra e imprime o `-e` literal; o **Bash** faz o inverso; com `shopt -s xpg_echo`, o Bash vira dash.

`valor="-n"; echo "$valor"`
Imprime **nada** — o valor virou opção. Qualquer valor vindo de fora que comece com hífen é engolido sem aviso.

`printf "$entrada"`
Dado **nunca** entra na cadeia de formato: um `%d%d%d` no valor consome argumentos que não existem. O certo é `printf '%s\n' "$entrada"`.

`printf '%b\n' "$txt"`
O substituto seguro de `echo -e`: interpreta a barra invertida do *argumento*, não do formato.

`%` `-` `10` `.2` `f`

`-` `sinalizador` — alinha à esquerda (padrão: direita)

`10` `largura` — ocupa 10 colunas, completa com espaço

`.2` `precisão` — 2 casas decimais

`f` `conversor` — ponto flutuante

A cadeia é reaproveitada ciclicamente até acabarem os argumentos: **`printf '%-8s %5.1f\n' raiz 42.75 var 8.5 home 120.4`** imprime **três linhas** sem laço nenhum — e é por isso que **`printf '%s\n' "$@"`** põe cada argumento na sua linha.

`%s` texto · **`%d`** inteiro · **`%.2f`** decimal · **`%x`** hex · **`%o`** octal (permissões!) · **`%b`** com barras · **`%q`** escapado para o shell · **`%%`** literal.

O IDIOMA CANÔNICO DO LAÇO DE LEITURA — DECORE O BLOCO INTEIRO

```
while IFS= read -r linha || [[ -n "$linha" ]] ; do … done < "$arq"
```

`IFS=`
Esvazia o separador **só nesta linha**. Sem ele, o `read` apara espaços e tabulações das pontas — e a linha indentada perde a indentação.

`-r`
Desliga o tratamento da barra invertida. Sem ele, `C:\Users` vira `C:Users` e uma linha terminada em `\` é colada na seguinte.

`|| [[ -n "$linha" ]]`
Salva a **última linha** quando o arquivo não termina em quebra — caso em que o `read` lê o conteúdo e mesmo assim retorna falha.

`< "$arq"`
Redirecionamento, não *pipe*: `cat arq | while` funciona, mas cria **subshell** e as variáveis alteradas no laço voltam vazias.

As três formas erradas mais comuns: `for linha in $(cat arq)` quebra em **espaços**, não em linhas · `cat arq | while read linha` subshell + perde indentação · `while read linha` come barra invertida e espaços. E para dividir campos, o `IFS` trabalha a favor: `while IFS=: read -r usuario _ uid _ _ _ shell; do … done < /etc/passwd` — o `IFS=:` vale só para aquele `read`, o do script fica intacto.

Senha lida com `read -s` continua exposta. O `-s` resolve o ombro do colega e mais nada: o valor fica numa variável do shell (legível pelo root), **aparece inteiro no rastreio se o script rodar com `set -x`**, e some para o `ps aux` de qualquer usuário se você o passar adiante como argumento. O caminho correto: `read -rs` seguido de entrada padrão — `printf '%s' "$senha" | comando --password-stdin` — ou um gerenciador como o `pass`; e `set +x` antes de qualquer linha que toque na variável. Ao confirmar ação irreversível, o padrão é **não**: `read -rp "Apagar? [s/N] " r` com `${r,,}` para comparar em minúsculas, e `rm -rf -- "$ALVO"` — o `--` encerra as opções e protege contra caminho iniciado por hífen. Sem terminal (`[[ ! -t 0 ]]`), aborte com mensagem: no `cron`, um `read` sem `-t` espera para sempre.

⚡ RESUMO RÁPIDO

- ✓ `echo` **só para texto literal simples.** Bash, dash e `/bin/echo` discordam sobre `-e`, e valor começado com hífen vira opção.
- ✓ `printf` **é o substituto em qualquer shell**, e a cadeia se reaproveita: `printf '%s\n' "$@"` e as tabelas com `%-20s` saem em uma linha.
- ✓ **Dado nunca entra na cadeia de formato** — sempre `printf`

🐉 NO KALI

Lista de alvos é o uso nº 1 do `read` em laboratório — e o que mais quebra. Um `alvos.txt` real tem linha em branco, comentário com `#`, espaço no fim e, se veio de planilha, terminador **CRLF**. Limpe no começo do laço: `linha="${linha%$'\r'}"` e `[[ -z "${linha// }" || "$linha" == \#* ]] && continue`. Apontar a ferramenta para um alvo com um retorno de carro colado no nome é como um teste autorizado vira acesso não autorizado — e a Lei